

P02

Aprender representaciones retropropagando errores

Learning representations by back-propagating errors

David E. Rumelhart, Geoffrey E. Hinton, Ronald J. Williams · 1986 · Nature, 323, 533–536

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P02 — Backpropagation

El procedimiento que hizo entrenables las capas ocultas: la red deja de recibir las representaciones y empieza a descubrirlas.

Nivel: L2 · Motor: `backprop` · Notebook: `P02_backpropagation.ipynb` · Anexo: cálculo y gradientes

1. Identificación

Campo	Valor
Título original	<i>Learning representations by back-propagating errors</i>
Autoría	David E. Rumelhart, Geoffrey E. Hinton, Ronald J. Williams
Año	1986
Venue	<i>Nature</i> , 323, 533–536
Fuente primaria	doi.org/10.1038/323533a0
Acceso	Restringido (revista de suscripción)
Fecha de consulta	2026-08-16

2. Problema anterior

El perceptrón solo traza fronteras lineales. La solución evidente —apilar capas— chocaba con un obstáculo práctico: **¿cuánta culpa del error final tiene un peso de la capa intermedia?** Ese peso no está conectado directamente con la salida; su efecto pasa por todas las unidades que vienen después.

Se le llamó el *credit assignment problem*. Sin resolverlo, las capas ocultas eran una idea sin algoritmo: se podían dibujar, no entrenar.

3. Propuesta

Aplicar la **regla de la cadena** recorriendo la red hacia atrás:

- propagación hacia adelante: se calcula la salida y la pérdida;
- propagación hacia atrás: el error de cada capa se expresa en función del error de la siguiente, multiplicando por los pesos y por la derivada de la activación;
- actualización: cada peso se mueve en la dirección opuesta a su gradiente.

La aportación decisiva del artículo no es la fórmula —ya existía en otras formulaciones— sino **mostrar que funciona en la práctica y que las unidades ocultas aprenden representaciones internas útiles y no diseñadas por nadie**. El título lo dice: *learning representations*.

4. Intuición sin fórmulas

Una cadena de montaje produce una pieza defectuosa. El responsable de calidad no reparte la culpa al azar: retrocede puesto por puesto preguntando «¿cuánto habría cambiado el defecto si tú hubieras hecho tu parte un poco distinta?». Cada puesto recibe una responsabilidad proporcional a su influencia real.

Dónde deja de funcionar la analogía: la cadena de montaje tiene un número fijo de puestos con influencia comparable. En una red profunda, la influencia se multiplica en cada paso, y ese producto puede colapsar a cero o dispararse — el problema que Po3 tendrá que atacar.

5. Matemática mínima

Red $x \rightarrow h = \sigma(w_1x + b_1) \rightarrow o = \sigma(w_2h + b_2)$, pérdida $L = (o - y)^2$, con $\sigma(z) = 1/(1+e^{-z})$ y $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

```
delta_o = dL/dO_in = 2(o - y) * sigma'(O_in)

dL/dW2 = delta_o * h^T
dL/dB2 = delta_o

delta_h = (W2^T delta_o) o sigma'(h_in)      <- el error "viaja" hacia atrás

dL/dW1 = delta_h * x^T
dL/dB1 = delta_h

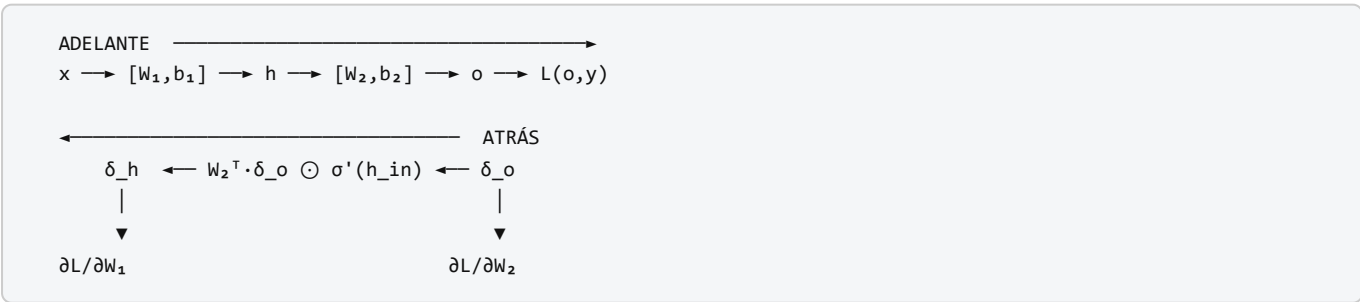
Actualización: theta <- theta - eta * dL/dtheta
```

El único ingrediente nuevo respecto al cálculo elemental es el orden: computar δ de atrás hacia adelante evita recalcular derivadas repetidas. Eso es lo que lo hace viable.

[!TIP] **Puente matemático.** Esta sección da por sabido lo siguiente. Si algo no te suena, léelo primero: está explicado una sola vez, en un solo sitio, y sirve para todas las fichas.

Dónde	Qué necesitas de ahí
A03 §2 · Regla de la cadena	la regla de la cadena es el algoritmo: todo lo demás es contabilidad
A03 §3 · Retropropagación	la retropropagación como aplicación ordenada de esa regla, capa por capa

6. Arquitectura o flujo



7. Qué observar en el paper original

- El artículo de *Nature* es **muy breve** (4 páginas). La formulación extendida está en el capítulo 8 de *Parallel Distributed Processing* (Rumelhart, Hinton y Williams, 1986).
- El experimento de las **representaciones internas**: la red descubre codificaciones en las unidades ocultas que corresponden a regularidades del dominio (por ejemplo, simetría o relaciones familiares) sin que se las especifique.
- Lo que **no** aparece: ReLU, dropout, normalización por lotes, Adam, inicialización cuidadosa, GPU. Todo eso es posterior y es lo que hizo escalar el método décadas después.

8. Evidencia y resultados

El paper demuestra, con problemas pequeños, que una red multicapa entrenada así:

- resuelve tareas no linealmente separables;
- **desarrolla representaciones internas interpretables** en las unidades ocultas.

Los problemas del artículo son de juguete para los estándares actuales; su valor es de **existencia** («esto se puede entrenar»), no de escala. Verificar las figuras concretas en la fuente primaria antes de citar cualquier resultado numérico.

La miniatura de este eje produce evidencia complementaria y verificable: XOR resuelto con 9 parámetros, y comprobación numérica del gradiente con error $\approx 1e-8$.

9. Impacto

- Desbloquea las redes multicapa y cierra el invierno abierto por el análisis de 1969.
- Es el algoritmo que **sigue entrenando todo hoy**: cada modelo del resto de esta ruta — LSTM, AlexNet, Transformer, GPT — se entrena con retropropagación.
- Establece la idea de *representación aprendida*, que es el concepto central del deep learning y da nombre a una conferencia entera (ICLR).

10. Limitaciones

1. **Gradiente desvaneciente o explosivo.** Al encadenar muchas capas, el producto de derivadas colapsa o diverge. El paper no lo aborda; será el tema de PO3.
2. **Óptimos locales y mesetas.** No hay garantía de convergencia al mínimo global.
3. **Coste de memoria.** Hay que guardar las activaciones de la pasada hacia adelante.
4. **Sensibilidad a la inicialización y a la tasa de aprendizaje.** Con sigmoides y pesos mal escalados, el entrenamiento se estanca en la zona de saturación.
5. **Poca plausibilidad biológica.** El cerebro no dispone de un canal simétrico que transporte el error hacia atrás con los mismos pesos (*weight transport problem*).
6. **Requiere activaciones diferenciables**, lo que excluye la función escalón de PO1.

11. Errores comunes

Error	Corrección
«Rumelhart, Hinton y Williams inventaron la retropropagación»	La popularizaron para redes neuronales . La diferenciación en modo reverso es anterior: Linnainmaa (1970), Werbos (1974). El propio artículo reconoce trabajo previo.
«Backpropagation es un algoritmo de optimización»	Es un algoritmo para calcular gradientes . Quien optimiza es SGD, Adam u otro.
«Backprop garantiza encontrar la mejor solución»	Encuentra un punto donde el gradiente se anula, no el óptimo global.
«El paper usa ReLU»	No. Usa sigmoides. ReLU se generaliza a partir de 2010–2012.
«Basta con más capas»	Sin las técnicas posteriores (inicialización, normalización, residuales), más capas empeoran el entrenamiento.

12. Relación con trabajos anteriores

- **P01 Perceptrón (1958)** — el límite lineal que motiva todo.
- **Linnainmaa (1970)** — diferenciación automática en modo reverso. doi.org/10.1007/BF01931367
- **Werbos (1974)** — tesis doctoral que aplica la idea a modelos de ciencias sociales.
- **Minsky y Papert (1969)** — el análisis que definió el problema a superar.

13. Relación con trabajos posteriores

- **P03 LSTM (1997)** — ataca el gradiente desvaneciente en secuencias.
- **P04 AlexNet (2012)** — demuestra que el método escala con GPU, ReLU, dropout y datos masivos.
- **Glorot y Bengio (2010), He et al. (2015)** — inicialización que evita la saturación.
- **Autograd moderno** (Theano, TensorFlow, PyTorch, JAX) — la generalización del método a grafos de cómputo arbitrarios.

14. Notebook asociado

P02_backpropagation.ipynb

Qué implementa: una red 2-2-1 con sigmoides entrenada sobre XOR, con gradientes derivados **a mano** (sin autograd) y verificación numérica del gradiente.

Qué NO implementa: el experimento de representaciones internas del paper, ni ninguna red de tamaño realista.

```
ai-evolution paper-lab P02 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la expresión de δ_h en función de δ_o y explica cada factor.
Explicar	Explica por qué se calcula δ de atrás hacia adelante y no al revés.
Aplicar	Cambia la sigmoide por $\tanh$ en el notebook y ajusta la derivada correspondiente.
Analizar	Calcula el producto de derivadas de sigmoide en 10 capas suponiendo $\sigma' \approx 0,25$. ¿Qué queda del gradiente?
Evaluar	Un modelo entrena y la pérdida baja, pero la verificación numérica del gradiente da $1e-2$ de diferencia. ¿Confías en el entrenamiento? Justifica.
Crear	Implementa la comprobación de gradiente como una función reutilizable que reciba cualquier red y devuelva el error relativo máximo.

16. Autoevaluación

1. ¿Por qué la retropropagación es eficiente frente a calcular cada derivada parcial por separado?
2. ¿Qué papel juega $\sigma'(h_{in})$ en la propagación del error hacia atrás?
3. ¿Por qué $\sigma' \leq 0,25$ es una mala noticia en redes profundas?
4. ¿Qué diferencia hay entre backpropagation y descenso de gradiente estocástico?
5. ¿Por qué la verificación numérica del gradiente usa diferencia centrada y no hacia adelante?
6. ¿Qué ocurre si inicializas todos los pesos a cero? ¿Y todos al mismo valor no nulo?
7. ¿Qué idea del deep learning moderno **no** está en este paper y suele atribuírsele?

17. Respuestas esperadas

1. Porque reutiliza los δ ya calculados: el coste es proporcional al de una pasada hacia adelante, en lugar de una evaluación por parámetro.
2. Mide la sensibilidad local de la unidad. Si la neurona está saturada, $\sigma' \approx 0$ y el error deja de propagarse por esa ruta.
3. Porque los factores se multiplican: $0,25^{10} \approx 1e-6$. Las capas iniciales reciben una señal despreciable y dejan de aprender.
4. Backprop **calcula** el gradiente; SGD **usa** ese gradiente para actualizar. Son piezas distintas y se pueden sustituir por separado.
5. Porque su error es $O(\epsilon^2)$ frente a $O(\epsilon)$ de la diferencia hacia adelante: da varios dígitos más de precisión con el mismo número de evaluaciones.
6. Con todos a cero (o todos iguales), todas las unidades ocultas reciben el mismo gradiente y permanecen idénticas para siempre: la simetría no se rompe y la capa oculta es inútil.
7. ReLU, dropout, normalización por lotes, Adam, conexiones residuales, GPU. Todo posterior.

18. Fuentes primarias

- Rumelhart, D. E., Hinton, G. E. y Williams, R. J. (1986). *Learning representations by back-propagating errors*. **Nature**, 323, 533–536. doi.org/10.1038/323533a0 · consultado 2026-08-16.
- Linnainmaa, S. (1976). *Taylor expansion of the accumulated rounding error*. **BIT Numerical Mathematics**, 16, 146–160. doi.org/10.1007/BF01931367 · consultado 2026-08-16.

- Werbos, P. (1974). *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. Tesis doctoral, Universidad de Harvard.



Evaluación — P02 · Aprender representaciones retropropagando errores

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Learning representations by back-propagating errors* (1986, Nature, 323, 533–536) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P02_backpropagation.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).